

Individual Contribution Reflection - Tom Tian

COMP4347 / COMP5347 Assignment 2 - MERN Quiz Game | Primary subsystem: Integration, validation, robustness, shared shell, documentation, and tests

Subsystem ownership. My primary responsibility was Subsystem D: the cross-cutting integration layer that connected the independently owned auth, quiz, admin, and Review Mode work into a coherent MERN application. I implemented and hardened shared Express app wiring, the response envelope, error handling, rate-limit responses, Swagger/Postman documentation, frontend API integration, theme/shared shell behaviour, and regression tests. I deliberately avoided taking over Auth (Tracy), Quiz/Review/Leaderboard business logic (Raven), or Admin CRUD (Allen), but I verified their contracts where they crossed subsystem boundaries.

Implementation and Rubric Alignment

The assignment required a clean MERN architecture, consistent API responses, protected routes, rate limiting, React Context state, dark mode, documentation, and evidence-backed individual contribution. My work targeted those integration and robustness marks rather than adding speculative features.

- **Backend integration:** ``backend/src/app.js`` wires security middleware, route mounting, Swagger, 404 handling, and the central ``errorHandler``.
- **Response contract:** ``backend/src/utils/responseEnvelope.js`` provides ``{ success, data?, error? }``, and tests cover both success and failure paths.
- **Validation and robustness:** rate-limit handlers return envelope-compatible ``429`` responses; final Swagger fixes align route middleware with documented ``401/403/429`` outcomes.
- **Frontend shell:** ``frontend/src/App.jsx``, ``api/api.js``, ``main.jsx``, and ``ThemeContext.jsx`` integrate routing, API calls, protected UI flow, and persisted theme behaviour.
- **Documentation and QA:** Swagger, Postman, README, and Jest/Supertest tests make the delivered behaviour auditable by markers.

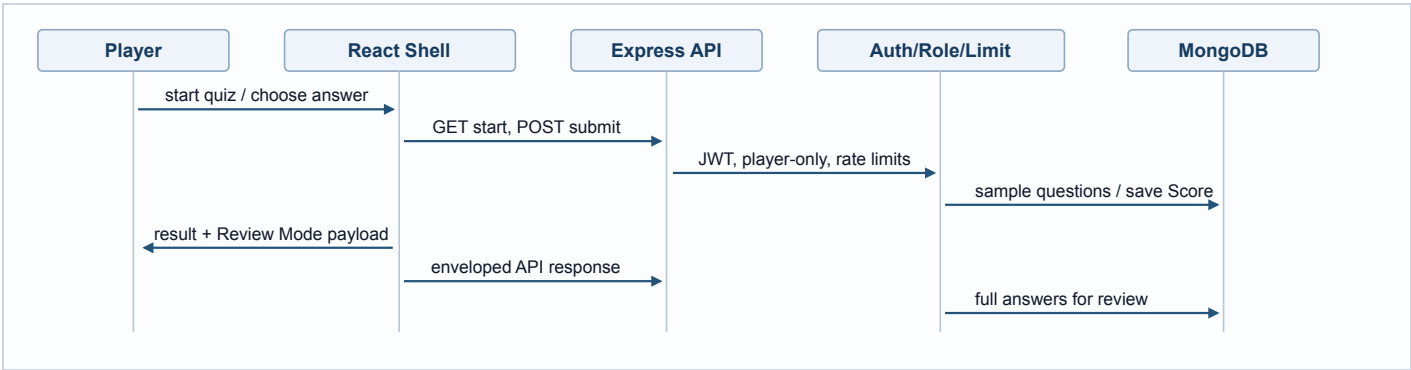
Major Technical Challenge

The hardest integration issue was maintaining one answer contract across shuffled quiz options, scoring, persistence, and Review Mode. ``/api/quiz/start`` returns shuffled options without ``correctAnswer``; the frontend submits the visible option index; the backend must score against the same deterministic shuffled order; and Review Mode must display selected and correct answers in that same order.

I found a test that still assumed raw ``correctAnswer === 0``, which could create false failures or hide real drift. I corrected ``backend/src/tests/api.test.js`` so the test first calls ``/api/quiz/start``, locates the correct visible option, submits through ``/api/quiz/submit``, and then checks the saved review payload. Commit ``bba8b06`` is the direct evidence.

This resolution supports the approved variation without changing quiz scoring rules: it validates the Review Mode contract rather than inventing a new mechanic.

Subsystem Sequence Diagram



Commit History Analysis

The following commits show my contribution pattern: first establishing shared infrastructure, then adding API documentation and validation evidence, then hardening final submission contracts. The hashes are intentionally specific so the marker can verify the work in GitHub Enterprise.

Hash	Commit	Reflection on Contribution
b176192	workspace scaffolding	Set up the project structure that allowed backend, frontend, scripts, and shared checks to evolve together.
90bf1d2	scaffold dependencies	Aligned root/backend/frontend scripts and demo tooling so reviewers could install, run, test, and build consistently.
b6d0ffe	backend foundation	Established the Express foundation used by feature owners and later integration tests.
57ebccb	shared rate limiters	Added reusable login and quiz-submit throttling with envelope-compatible failure responses.
d7c20c6	frontend providers	Integrated shared frontend providers needed for routing, auth-aware API use, and theme behaviour.
51c7e39	shared app shell	Created the application shell tying public, player, and admin flows into one navigable React app.
8501ac2	Swagger docs	Added self-hosted API documentation required by the assignment and useful for cross-team contract review.
7acd98d	envelope/error tests	Verified the response envelope and error handler rather than relying on manual inspection.
03a5d8e	architecture docs	Documented integration decisions and system architecture for marker review and team alignment.
2bda2bb	security/API evidence	Recorded validation and security evidence around JWT, rate limiting, and API behaviours.
6686eae	D docs/tests	Expanded seed, API, Postman, and regression coverage around the shared delivery surface.
972fc77	final QA alignment	Reconciled D-owned tests and documentation with the actual API surface before review.
bba8b06	shuffled answer tests	Fixed a test-contract bug so Review Mode scoring and review assertions match visible shuffled options.
4a8fe22	leaderboard API docs	Aligned Swagger with the authenticated, player-only leaderboard route.
d0d07f6	quiz error responses	Completed route/middleware/Swagger response consistency for `403` and `429` outcomes.

Review Mode Design Reflection

Review Mode was a good variation for this project because it improves the educational value of the quiz without changing the core scoring rule of +1 per correct answer. My integration work supported this decision by ensuring the completed attempt stores enough answer data to reconstruct what the player saw and selected.

The main design risk was contract drift: if shuffled options, submitted indexes, saved answers, and review rendering disagreed, Review Mode would look correct while explaining the wrong answer. The final API test and Swagger fixes were therefore not cosmetic; they made the variation auditable and protected the correctness of the learning feedback loop.

Final Verification Evidence

- `npm test --prefix backend` passed: 6 suites / 22 tests.
- `npm run build --prefix frontend` passed.
- `python3 -m json.tool docs/postman-collection.json >/dev/null` passed.
- `git diff --check` passed.
- Root, backend, and frontend `npm audit` reported 0 vulnerabilities.

Overall, my contribution is best understood as integration ownership: I made independent feature slices testable, documented, consistent, and submission-ready while preserving clear ownership boundaries.